
嵌入式系统实验指导书

试验 2

2025 年 9 月

目录

目录	2
1. 实验二：串口实验	3
1.1. 串口实验	3
1.1.1. 实验目的.....	3
1.1.2. 实验环境.....	3
1.1.3. 实验原理.....	3
1.1.4. 实验内容.....	7
1.1.5. 实验步骤.....	9
1.1.6. 实验现象.....	14
1.1.7. 思考题.....	14

1. 实验二：串口实验

1.1. 串口实验

1.1.1. 实验目的

- 1) 了解并掌握 stm32 的串口的使用
- 2) 掌握将 printf 的打印和 scanf 的输入重定向到串口以实现串口输入和打印的方法

1.1.2. 实验环境

- 1) 硬件：STM32 开发板、程序下载调试板、ARM JLINK 仿真器、交叉串口线、USB 方口线、DC 5V 电源和 PC 机。

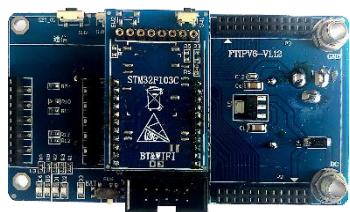


图 1-1 STM32 开发板

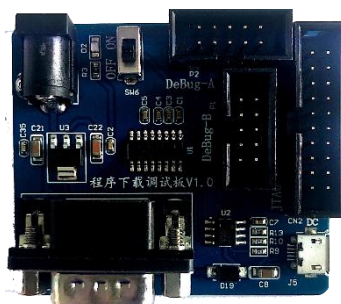


图 1-2 程序下载调试板



图 1-3 ARM JLINK 仿真器

- 2) 软件：Windows7 及以上系统、Keil5 MDK-ARM5.43a、串口调试助手 SuperCom。

1.1.3. 实验原理

STM32 开发板上没有 DB9 口，需要通过 10 针排线连接到下载调试板，下载调试板上有 DB9 口。由原理图可以看出，下载调试板的串口 RX、TX 通过 10 针排线连接到 STM32 的 PA9 和 PA10，这是 STM32 的串口 1。原理图如下图所示。

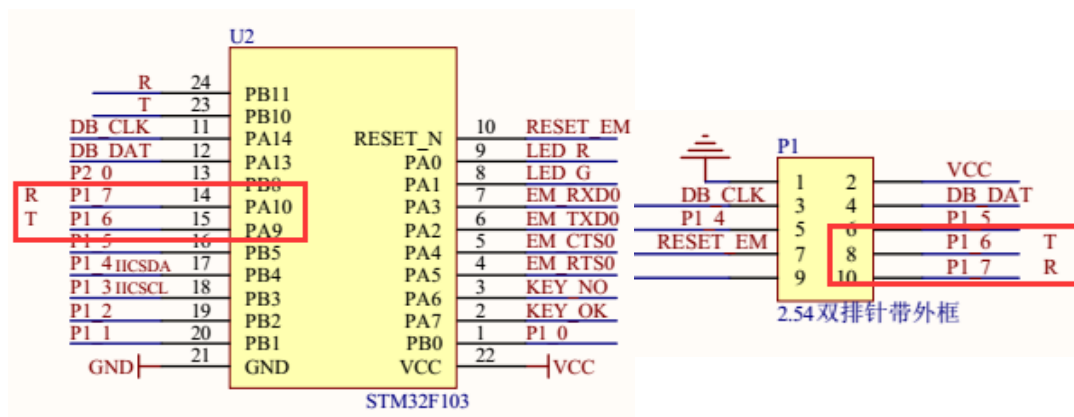


图 1-4 STM32 串口引脚图

本次实验使用的是串口 1，串口 1 对应的 STM32 的 IO 引脚为：

PA9-----TX

PA10-----RX

串口模式的操作具有以下特点：

8 位或者 9 位的负载数据

奇偶校验或者无奇偶校验

配置起始位或者停止位电平

独立收发中断

接下来，我们先简单介绍下几个与串口基本配置直接相关的固件库函数。这些函数和定义主要分布在 `stm32f10x_usart.h` 和 `stm32f10x_usart.c` 文件中。

1) 串口时钟使能。串口是挂载在 APB2 下面的外设，所以使能函数为：

```
RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1);
```

2) 串口复位。当外设出现异常的时候可以通过复位设置，实现该外设的复位，然后重新配置这个外设达到让其重新工作的目的。一般在系统刚开始配置外设的时候，都会先执行复位该外设的操作。复位是在函数 `USART_DeInit()` 中完成的：

```
void USART_DeInit(USART_TypeDef*USARTx);//串口复位
```

比如我们要复位串口 1，方法为：

```
USART_DeInit(USART1);//复位串口 1
```

3) 串口参数初始化。串口初始化是通过 `USART_Init()` 函数实现的，

```
void USART_Init(USART_TypeDef*USARTx,USART_InitTypeDef*USART_InitStruct);
```

这个函数的第一个入口参数是指定初始化的串口标号，这里选择 `USART1`。第二个入口参数是一个 `USART_InitTypeDef` 类型的结构体指针，这个结构体指针的成员变量用来设置串口的一些参数。一般的实现格式为：

```

USART_InitStructure.USART_BaudRate = bound; //波特率设置;
USART_InitStructure.USART_WordLength = USART_WordLength_8b ;//字长为 8 位数据格式
USART_InitStructure.USART_StopBits = USART_StopBits_1; //一个停止位
USART_InitStructure.USART_Parity = USART_Parity_No; //无奇偶校验位
USART_InitStructure.USART_HardwareFlowControl
= USART_HardwareFlowControl_None; //无硬件数据流控制
USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx; //收发模式
USART_Init(USART1, &USART_InitStructure); //初始化串口

```

从上面的初始化格式可以看出初始化需要设置的参数为：波特率，字长，停止位，奇偶校验位，硬件数据流控制，模式（收，发）。我们可以根据需要设置这些参数。

4) 数据发送与接收。STM32 的发送与接收是通过数据寄存器 USART_DR 来实现的，这是一个双寄存器，包含了 TDR 和 RDR。当向该寄存器写数据的时候，串口就会自动发送，当收到数据的时候，也是存在该寄存器内。STM32 库函数操作 USART_DR 寄存器发送数据的函数是：

```
void USART_SendData(USART_TypeDef*USARTx,uint16_t Data);
```

通过该函数向串口寄存器 USART_DR 写入一个数据。

STM32 库函数操作 USART_DR 寄存器读取串口接收到的数据的函数是：

```
uint16_t USART_ReceiveData(USART_TypeDef*USARTx);
```

通过该函数可以读取串口接受到的数据。

5) 串口状态。串口的状态可以通过状态寄存器 USART_SR 读取。USART_SR 的各位描述如图 1-5 所示：



图 1-5 USART_SR 寄存器各位描述

这里我们关注一下两个位，第 5、6 位 RXNE 和 TC。RXNE（读数据寄存器非空），当该位被置 1 的时候，就是提示已经有数据被接收到了，并且可以读出来了。这时候我们要做的就是尽快去读取 USART_DR，通过读 USART_DR 可以将该位清零，也可以向该位写 0，直接清除。TC（发送完成），当该位被置位的时候，表示 USART_DR 内的数据已经被发送完成了。如果设置了这个位的中断，则会产生中断。该位也有两种清零方式：

读 USART_SR，写 USART_DR。

直接向该位写 0。状态寄存器的其他位我们这里就不做过多讲解，大家需要可以查看中文参考手册。在我们固件库函数里面，读取串口状态的函数是：

```
FlagStatus USART_GetFlagStatus(USART_TypeDef*USARTx,uint16_t USART_FLAG);
```

这个函数的第二个入口参数非常关键，它是标示我们要查看串口的哪种状态，比如上面讲解的 RXNE(读数据寄存器非空)以及 TC(发送完成)。例如我们要判断读寄存器是否非空(RXNE)，操作库函数的方法是：

```
USART_GetFlagStatus(USART1,USART_FLAG_RXNE);
```

我们要判断发送是否完成(TC)，操作库函数的方法是：

```
USART_GetFlagStatus(USART1,USART_FLAG_TC);
```

这些标识号在 MDK 里面是通过宏定义定义的：

```
#define USART_IT_PE ((uint16_t)0x0028)
#define USART_IT_TXE ((uint16_t)0x0727)
#define USART_IT_TC ((uint16_t)0x0626)
#define USART_IT_RXNE ((uint16_t)0x0525)
#define USART_IT_IDLE ((uint16_t)0x0424)
#define USART_IT_LBD ((uint16_t)0x0846)
#define USART_IT_CTS ((uint16_t)0x096A)
#define USART_IT_ERR ((uint16_t)0x0060)
#define USART_IT_ORE ((uint16_t)0x0360)
#define USART_IT_NE ((uint16_t)0x0260)
#define USART_IT_FE ((uint16_t)0x0160)
```

6) 串口使能。串口使能是通过函数 USART_Cmd()来实现的，这个很容易理解，使用方法是：

```
USART_Cmd(USART1,ENABLE);//使能串口
```

7) 开启串口响应中断。有些时候当我们还需要开启串口中断，那么我们还需要使能串口中断，使能串口中断的函数是：

```
void USART_ITConfig(USART_TypeDef*USARTx,uint16_t USART_IT,FunctionalState NewState)
```

这个函数的第二个入口参数是标示使能串口的类型，也就是使能哪种中断，因为串口的中断类型有很多种。比如在接收到数据的时候（RXNE 读数据寄存器非空），我们要产生中断，那么我们开启中断的方法是：

```
USART_ITConfig(USART1,USART_IT_RXNE,ENABLE);//开启中断，接收到数据中断
```

我们在发送数据结束的时候（TC，发送完成）要产生中断，那么方法是：

```
USART_ITConfig(USART1, USART_IT_TC, ENABLE);
```

8) 获取相应中断状态。当我们使能了某个中断的时候，当该中断发生了，就会设置状态寄存器中的某个标志位。经常我们在中断处理函数中，要判断该中断是哪种中断，使用的函数是：

```
ITStatus USART_GetITStatus(USART_TypeDef*USARTx,uint16_t USART_IT)
```

比如我们使能了串口发送完成中断，那么当中断发生了，我们便可以在中断处理函数中调用这个函数来判断到底是否是串口发送完成中断，方法是：

```
USART_GetITStatus(USART1, USART_IT_TC)
```

返回值是 SET，说明是串口发送完成中断发生。

通过以上的讲解，我们就可以达到串口最基本的配置了，关于串口更详细的介绍，请参考《STM32 参考手册》的通用同步异步收发器一章。

1.1.4. 实验内容

本次实验以 STM32 单片机为控制载体，通过重定向 C 语言的输入输出函数实现串口的输入和打印功能。

1) 打开 usart.c 文件，配置串口 1 的初始化

```
/******  
*****/  
#include "usart.h"  
#include <stdio.h>  
void uart1_init()  
{  
    USART_InitTypeDef USART_InitStructure;  
    GPIO_InitTypeDef GPIO_InitStructure;  
  
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA | RCC_APB2Periph_USART1, ENABLE);  
  
    /* 配置串口 1 Tx (PA9) 为推挽复用模式 */  
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;  
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;  
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;  
    GPIO_Init(GPIOA, &GPIO_InitStructure);  
  
    /* 配置串口 1 Rx (PA10) 为浮空输入模式 */  
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;  
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;  
    GPIO_Init(GPIOA, &GPIO_InitStructure);  
  
    /* 配置串口 1 的各种参数 */  
    USART_InitStructure.USART_BaudRate = 115200;  
    USART_InitStructure.USART_WordLength = USART_WordLength_8b;  
    USART_InitStructure.USART_StopBits = USART_StopBits_1;  
    USART_InitStructure.USART_Parity = USART_Parity_No;  
    USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None;  
    USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx;  
  
    USART_Init(USART1, &USART_InitStructure);  
    /* 串口 1 开始工作 */  
    USART_Cmd(USART1, ENABLE);  
}
```

2) 重定向 fputc 函数，将 fputc 函数里的内容改为串口发送的内容，这样就可以调用 printf() 函数实现串口打印

```
int fputc(int ch, FILE *f) {  
    USART_SendData(USART1, (uint8_t)ch);  
    while (USART_GetFlagStatus(USART1, USART_FLAG_TXE) == RESET);  
}
```

```
    return ch;
}
```

3) 重定向 fgetc 函数，将 fgetc 函数里改为串口接收一个字节，这样就可以调用 scanf() 函数实现串口输入

```
int fgetc(FILE *f) {
    while (USART_GetFlagStatus(USART1, USART_FLAG_RXNE) == RESET) {
    }
    return (int)USART_ReceiveData(USART1);
}
```

4) 新建 usart.h

```
/******
*****/
#ifndef USART_H
#define USART_H

#include "stm32f10x_rcc.h"
#include "stm32f10x_gpio.h"
#include "stm32f10x_usart.h"
#include "misc.h"

void uart1_init(void);
#endif
```

5) 打开 main.c 文件，配置主函数

```
/******
*****/
#include <stdio.h>
#include <string.h>
#include "stm32f10x.h"
#include "usart.h"
int main(void)
{
    char ch;
    uart1_init();//串口 1 初始化

    printf("Stm32 example start !\r\n");

    while(1)
    {
        printf("please enter a charater:\r\n");
        scanf("%c", &ch);
        printf("the charater is %c\r\n", ch);
    }
    return 0;
}
```

接下来我们来看一下本次串口实验的流程图：

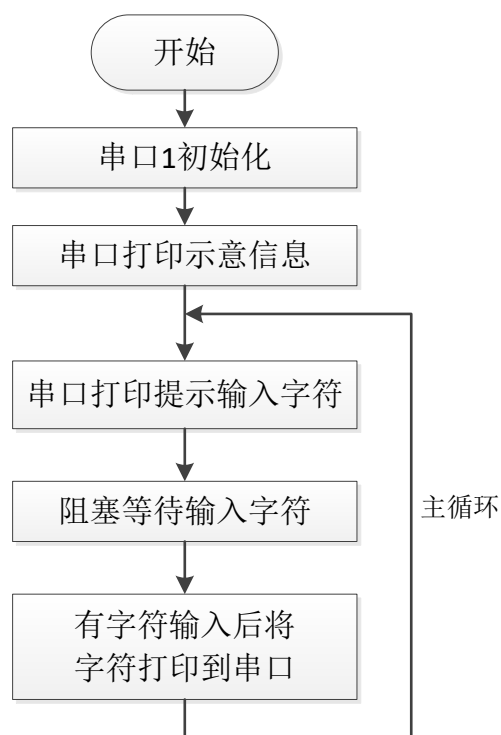


图 1-6 串口实验流程图

1.1.5. 实验步骤

- 1) 使用 USB 方口线连接 ARM J-LINK 仿真器和 PC 机，用 20 针排线连接 ARM J-LINK 仿真器和下载调试板，10 针排线连接下载调试板和 STM32 开发板，再使用串口线将下载调试板连接至 PC，使用 DC 5V 电源给开发板供电，如图 1-7 所示。

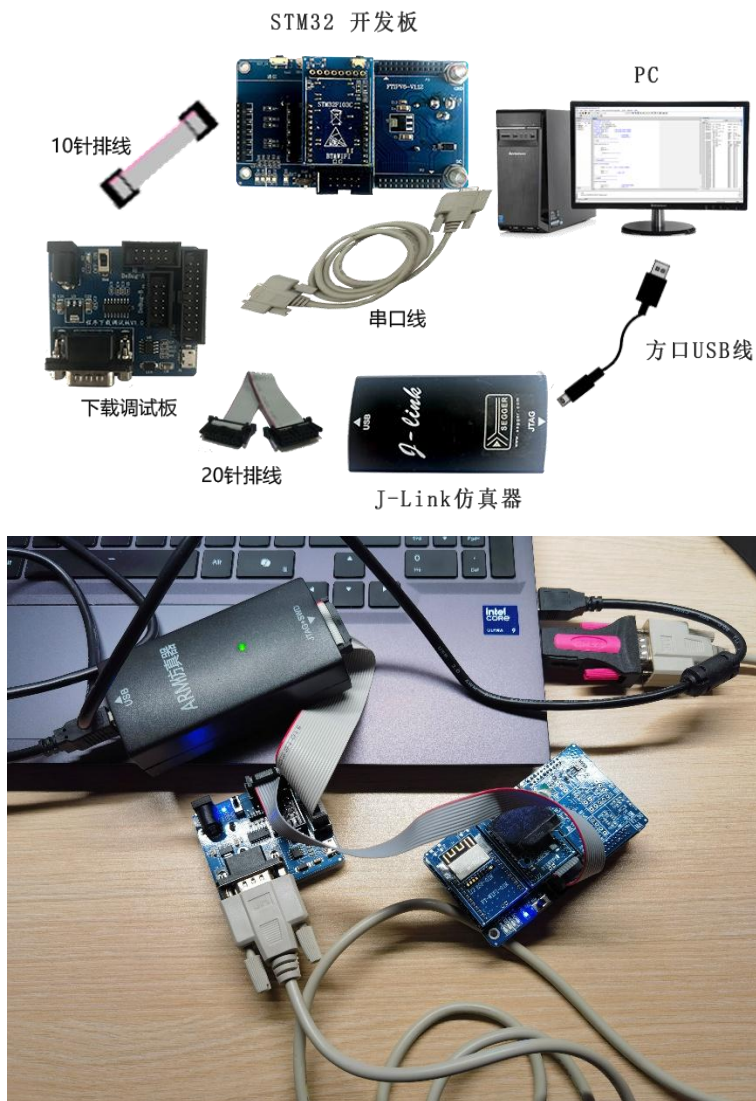


图 1-7 硬件连接图

- 2) 在 PC 机上查看 STM32 开发板连接到 PC 机的端口号，如果串口线连接到 PC 机的 DB9 口，通常端口号默认为 COM1，如果串口线是通过 U 转串连接到电脑 USB 口上的，则需要打开设备管理器查看正确的端口号，如图 1-8 所示，打开设备管理器（不同的 Windows 系统打开方式可能不同）。



图 1-8 打开设备管理器（左 win10，右 win11）

然后在设备管理器的端口下查看串口使用的端口号，如图所示。

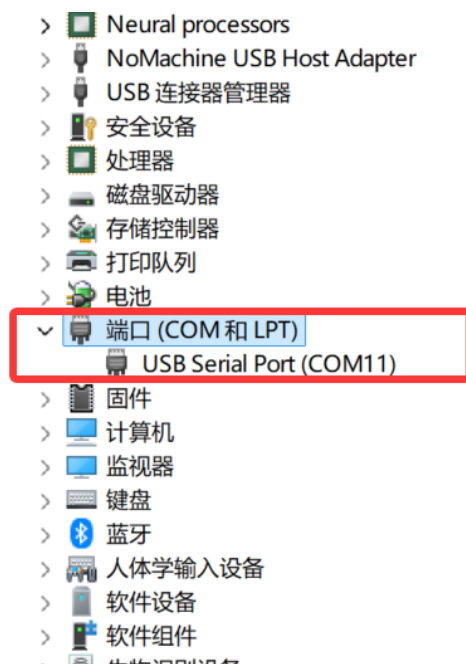


图 1-9 在设备管理器的端口下查看端口号

- 3) 用 Keil5 开发环境打开实验例程，点击 Rebuild All 重新编译工程，如图 1-10 所示。

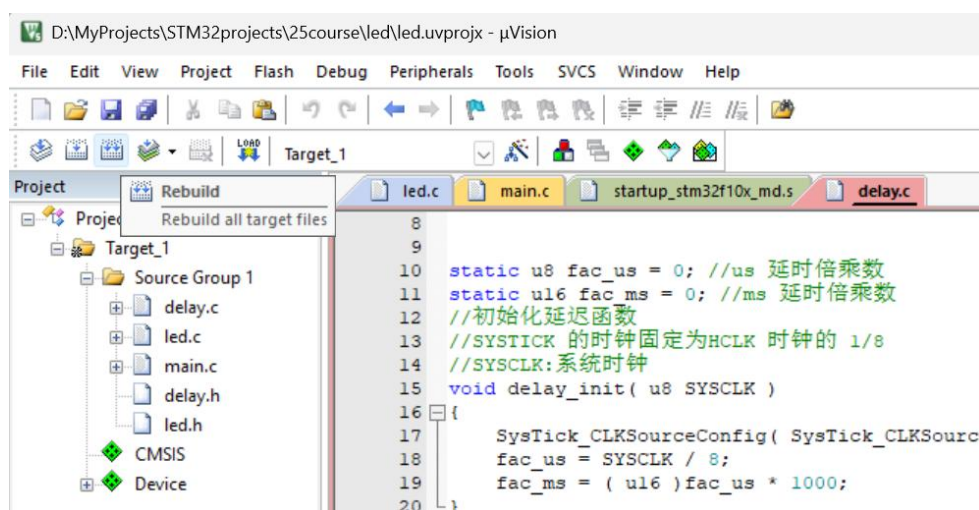


图 1-10 重新编译

- 4) 点击 Download 将程序下载到 STM32 开发板中，即可观察到开发板程序运行（如按照教程勾选“Reset and Run”）如图 1-11 所示。也可以将 STM32 开发板重新上电或者按下复位按钮让刚才下载的程序重新运行。

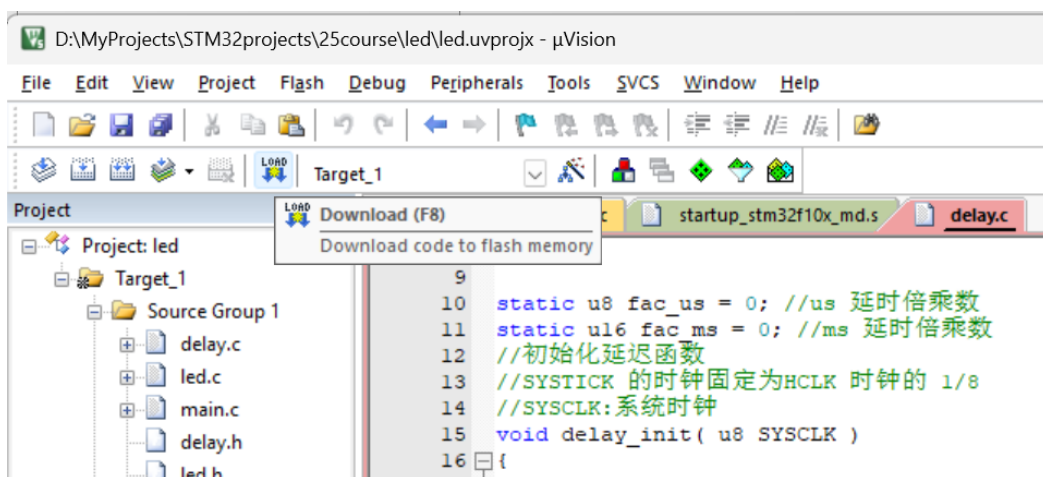


图 1-11 下载程序

- 5) （可选）下载完后可以点击 Debug 让程序进入调试模式，如图 1-12 所示；

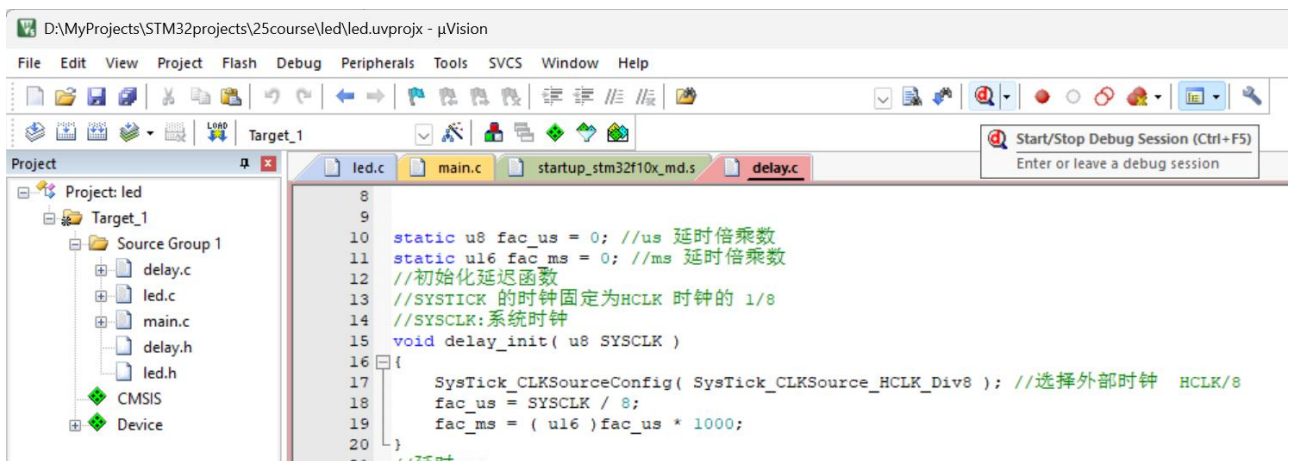


图 1-12 调试程序

- 6) 程序成功运行后，在 PC 机上打开 SuperCom.exe (直接解压压缩包，程序在解压后的文件夹内)，设置波特率为 115200，端口号选择步骤 2 查看到的端口，不勾选自动清空，其他设置采用默认值，点击打开串口，如图 1-13 所示。

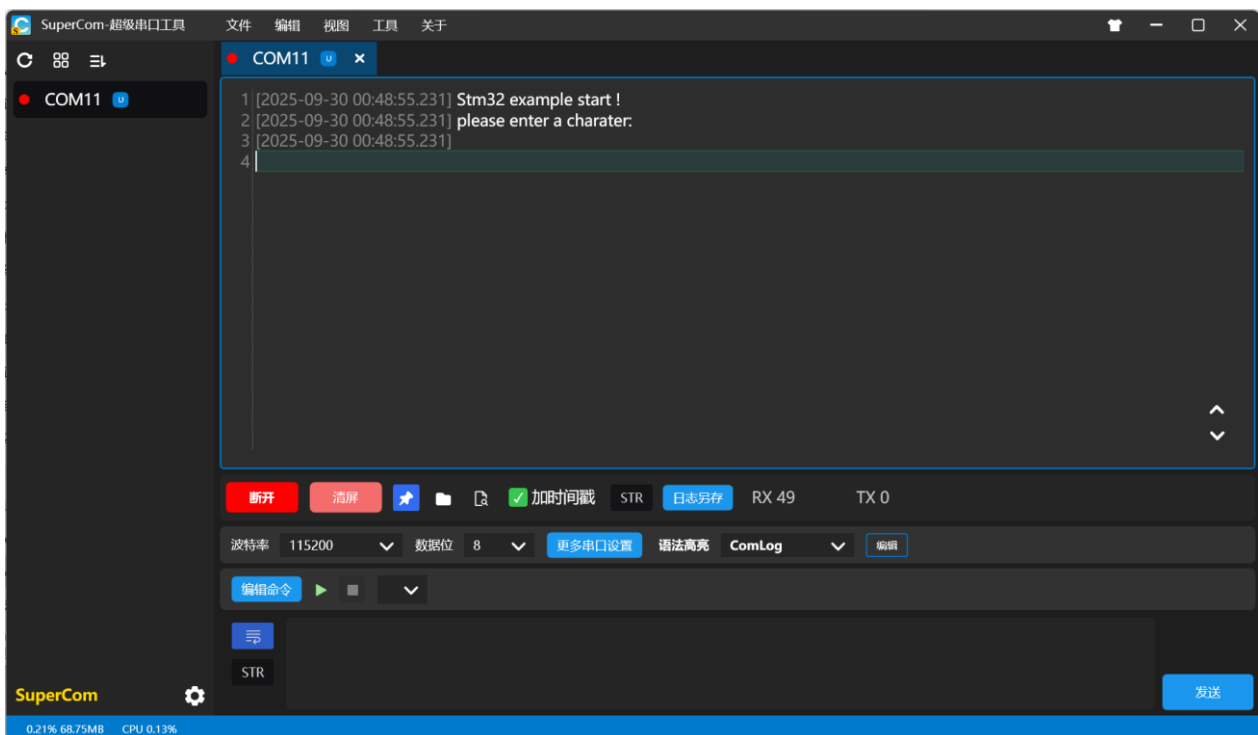


图 1-13 打开串口助手

- 7) 在发送区输入任意字符，点击手动发送，观察串口调试小助手接收区显示的数据。

1.1.6. 实验现象

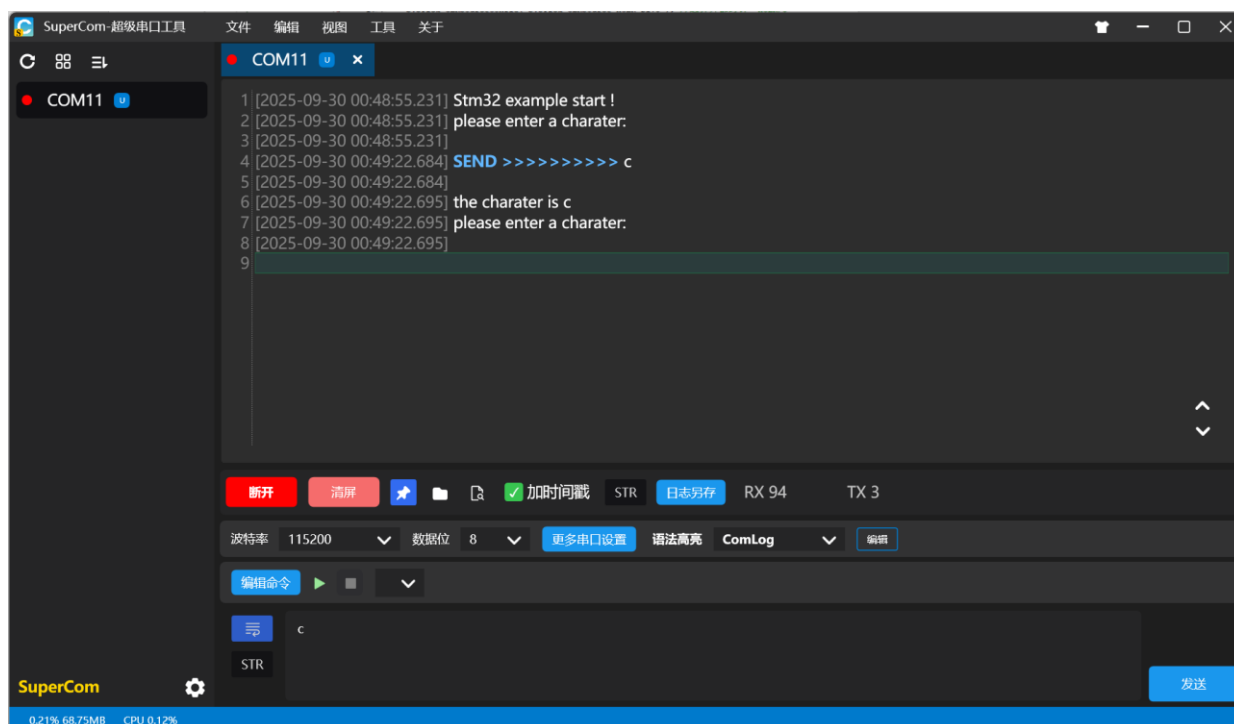


图 1-14 串口实验串口打印信息

串口调试小助手接收区收到所发送字符，如图 1-14 所示。

1.1.7. 思考题

若实现必须发送 5 到 10 个字符，之间的任意字符串，串口接收区才打印所发送字符串。代码需要如何实现呢？

请思考并动手修改代码，实现以上功能。

2. 附录：源代码

2.1. 串口实验

1) usart.c

```
/*
*****
*****
*/
#include "usart.h"
#include <stdio.h>

void uart1_init()
{
    USART_InitTypeDef USART_InitStructure;
    GPIO_InitTypeDef GPIO_InitStructure;
    RCC_APB2PeriphClockCmd( RCC_APB2Periph_GPIOA | RCC_APB2Periph_USART1, ENABLE );

    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_Init( GPIOA, &GPIO_InitStructure );

    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
    GPIO_Init( GPIOA, &GPIO_InitStructure );

    USART_InitStructure.USART_BaudRate = 115200;
    USART_InitStructure.USART_WordLength = USART_WordLength_8b;
    USART_InitStructure.USART_StopBits = USART_StopBits_1;
    USART_InitStructure.USART_Parity = USART_Parity_No;
    USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None;
    USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx;
    USART_Init( USART1, &USART_InitStructure );

    USART_Cmd( USART1, ENABLE );
}

int fputc(int ch, FILE *f) {
    USART_SendData(USART1, (uint8_t)ch);
    while (USART_GetFlagStatus(USART1, USART_FLAG_TXE) == RESET);
    return ch;
}

int fgetc(FILE *f) {
    while (USART_GetFlagStatus(USART1, USART_FLAG_RXNE) == RESET) {
    }
    return (int)USART_ReceiveData(USART1);
}
```

2) usart.h

```
/*
*****
*****
*/
```

```

#ifndef USART_H
#define USART_H

#include "stm32f10x_rcc.h"
#include "stm32f10x_gpio.h"
#include "stm32f10x_usart.h"
#include "misc.h"

void uart1_init(void);
#endif

```

3) main.c

```

/*****
*****/
#include <stdio.h>
#include <string.h>
#include "stm32f10x.h"
#include "usart.h"
int main(void)
{
    char ch;
    uart1_init();//串口 1 初始化

    printf("Stm32 example start !\r\n");

    while(1)
    {
        printf("please enter a charater:\r\n");
        scanf("%c", &ch);
        printf("the charater is %c\r\n", ch);
    }
    return 0;
}

```